Chapter 10: Delivering Food

Nature Inspired Optimisation for Delivery Problems (2022)

Chapter Overview

10.1 Introduction: Food Delivery in the Real World I

The COVID-19 pandemic fundamentally changed how food is distributed in many countries. Home delivery of food — from restaurants, supermarkets, and meal-kit companies — became a mainstream service almost overnight, requiring small businesses and community organisations to plan delivery routes that had previously been handled informally or not at all.

The Core Problem

A vehicle starts from a **depot**, must visit a set of customers, deliver food to each one within certain time and capacity constraints, and then return to the depot. The goal is to minimise the total effort (distance or time) while satisfying all constraints. This is a specialised form of the **Vehicle Routing Problem (VRP)**.

Unlike parcel delivery, food delivery has several characteristics that make it particularly challenging in practice:

Time-sensitive goods. Food quality degrades if it is not delivered within a reasonable time. This imposes a maximum round-trip time on each delivery vehicle (for example, 500 minutes per round in the experimental instance used in this chapter).

10.1 Introduction: Food Delivery in the Real World II

Variable demand. Different customers may order different quantities (measured in units such as "bag counts"). The vehicle has a finite *capacity* (maximum units it can carry), so it may need to make multiple rounds if the total demand exceeds that capacity.

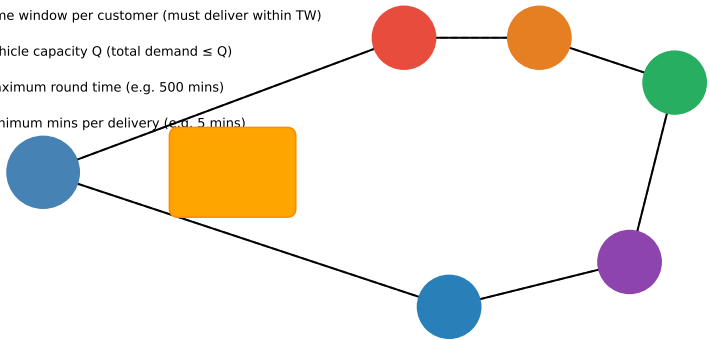
Small, inexperienced operators. The organisations deploying this software (such as volunteer food banks or small restaurants) may lack logistics expertise. The optimisation must be packaged as a simple-to-use application — not a research prototype.

A "bag" as the unit of delivery. The problem treats demand in discrete units called *bags*. Each customer requires a certain number of bags, and a vehicle can carry at most Q (Q) bags per round. Throughout the book the quantity Q denotes *vehicle capacity*: the maximum load (in bags) that can be loaded onto the vehicle at any one time.

Food Delivery Problem: Key Elements

Key constraints:

- Time window per customer (must deliver within TW)
- Vehicle capacity Q (total demand ≤ Q)
- Maximum round time (e.g. 500 mins)
- Minimum mins per delivery (e.g. 5 mins)



10.1 Introduction: Food Delivery in the Real World IV

Figure: Key elements of the food delivery problem. The vehicle leaves the Depot, visits customers A–E, and must return within the time limit. Both capacity Q (total bags) and the maximum transit time constrain which solutions are feasible.

Key Insight: Why This Problem Is Hard

Even for a modest fleet with 20–100 delivery stops, the number of possible orderings grows factorially. Exhaustive enumeration is completely impractical, so we rely on **nature-inspired (evolutionary) algorithms** to search the solution space efficiently.

10.2 Fitness: Choosing the Right Objective Function I

The *fitness function* (also called the *objective function*) is the mathematical formula that scores each candidate solution. A good fitness function does not merely measure *what* the algorithm does — it guides *how* the algorithm improves. Choosing the wrong fitness function can cause the algorithm to find technically correct but practically poor solutions.

10.2 Fitness: Choosing the Right Objective Function II

Candidate 1: Total Distance

The simplest approach is to sum the distance of every leg of the route:

$$F_{\text{dist}} = \sum_{i=1}^{n} d_i$$

where d_i ("d sub i") is the distance of the i-th leg and n (n) is the total number of legs in the route (including the final return leg to the depot). The index i runs from 1 to n, so $\sum$ (Sigma, meaning "sum over all legs") adds up all n distances.

Notation

- d_i — distance (e.g. in kilometres or minutes) of leg number i of the route.
- n — total number of legs: for a route visiting k customers, $n = k + 1$ (the return leg counts too).
- $\sum_{i=1}^{n}$ — Sigma: "add up the values $d_1, d_2, \ldots, d_n$ one by one".

10.2 Fitness: Choosing the Right Objective Function III

Why total distance is insufficient. Consider a route visiting customers A, B, C, D, E in some order, where customers care about *when* they are visited, not just whether they are visited. A route that drives 10 km to the furthest customer first and then collects several nearby customers on the way back scores the same total distance as a route that delivers to nearby customers first — yet customers served late in the route must wait much longer.

10.2 Fitness: Choosing the Right Objective Function IV

Candidate 2: Weighted Distance

The **weighted distance** (WD) assigns a weight to each leg equal to the number of customers who have *not yet been served* at the start of that leg:

$$WD = \sum_{i=1}^{n} d_i \times wc_i$$

where wc_i ("wc sub i", standing for *wait count*) is the number of remaining deliveries at the start of leg i.

10.2 Fitness: Choosing the Right Objective Function V

Notation

- d_i — distance of leg i (same as before).
- wc_i — "wait count" for leg i: the number of customers who have **not yet received** their delivery when the vehicle starts leg i. For a route with k customers, $wc_1 = k$, $wc_2 = k - 1$, $\ldots$, $wc_k = 1$, and the return leg $wc_{k+1} = 0$.
- $\sum_{i=1}^{n} d_i \times wc_i$ — sum of (distance of leg i) multiplied by (number of customers still waiting at the start of leg i).

How to Read This Formula

Every metre travelled while customers are still waiting "costs" more than metres travelled after all deliveries are complete. A long drive *early* in the route is penalised heavily because many customers are waiting; a long return drive *after* all deliveries are done has zero weight and costs nothing in WD terms. Minimising WD therefore pushes the algorithm towards **making deliveries as early as possible in the route**.

10.2 Fitness: Choosing the Right Objective Function VI

Worked Numerical Example. Suppose we have a route Depot $\to$ A $\to$ B $\to$ C $\to$ D $\to$ E $\to$ Depot with leg distances 2, 5, 6, 5, 3, 4 (total = 25). The wait counts are 5, 4, 3, 2, 1, 0 (decreasing by 1 after each delivery).

Worked Example: Computing WD

	Dep$\to$A	A$\to$B	B$\to$C	C$\to$D	D$\to$E	E$\to$Dep	Total
Distance d_i	2	5	6	5	3	4	25
Wait count wc_i	6	5	4	3	2	1	–
$d_i \times wc_i$	12	25	24	15	6	4	**86**

The **simple distance** fitness is 25. The **weighted distance** fitness is 86. These two numbers measure very different things: WD = 86 captures the total "waiting cost" accumulated across all customers, rewarding routes that serve customers quickly. To minimise WD, the algorithm should place *shorter* legs early in the route.

10.2 Fitness: Choosing the Right Objective Function VII

Weighted Distance Calculation: Route Depot→A→B→C→D→E→Depot

	Depot→A	A→B	B→C	C→D	D→E	E→Depot	Total
Distance	2	5	6	5	3	4	25
WC (wait count)	6	5	4	3	2	1	
WD = Dist × WC	12	25	24	15	6	4	86

WD = 86 vs simple distance = 25.
Minimising WD encourages early deliveries within the route.

Figure: Visual summary of the weighted-distance calculation. Each cell in the WD row is distance $\times$ wait count. The total WD (86) is much larger than total distance (25), reflecting the accumulated waiting cost.

10.2 Fitness: Choosing the Right Objective Function VIII

Weighted Distance Fitness Function

$$WD = \sum_{i=1}^{n} d_i \times wc_i$$

d_i = distance of the i-th leg of the route | wc_i = number of remaining deliveries when leg i is traversed

Behavioural insight: a long leg early in the route is penalised heavily because many deliveries still await. Placing short legs first minimises WD.

Figure: The weighted-distance formula. Long legs early in the route are penalised heavily because wc_i is large; by the time all customers have been served, $wc_i = 0$ and any return leg is free.

10.2 Fitness: Choosing the Right Objective Function IX

Three route strategies and their WD values. Given a set of customers, three natural strategies exist for ordering deliveries:

- **Route (a) — outward then return:** Visit customers from nearest to furthest, return directly. Simple distance is small, but WD may be large if the furthest customer is served very late.
- **Route (b) — furthest first:** Drive to the furthest point first, deliver on the way back. Some deliveries happen early; WD is lower.
- **Route (c) — mixed:** Make some deliveries on the outward journey and some on the return. This can achieve a good balance, and feedback from real users of the system confirmed that this pattern was most expected.

When the *time window* constraint is not tight, routes (b) and (c) are often equivalent in WD. When the time constraint *is* tight, the algorithm must find a feasible solution at all — any infeasible solution is penalised heavily.

10.2 Fitness: Choosing the Right Objective Function X

Fig. 10.1 Three Delivery Strategies for the Same Set of Customers

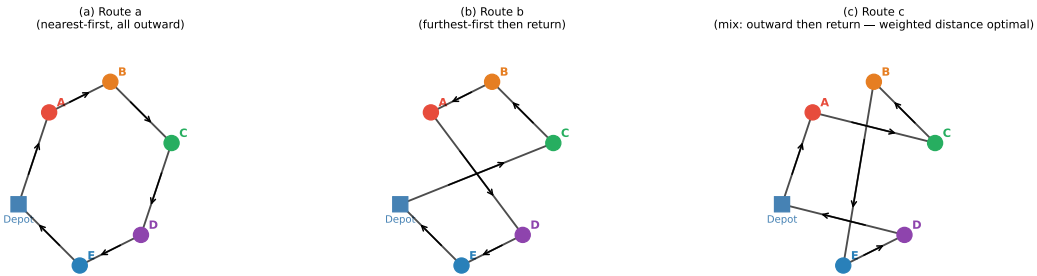


Figure: Three candidate delivery strategies for the same customer set. Route (a) delivers outward (nearest-first); route (b) delivers furthest-first; route (c) is a mixed approach. Minimising WD favours placing the shortest legs earliest, which often corresponds to route (b) or (c).

10.2 Fitness: Choosing the Right Objective Function XI

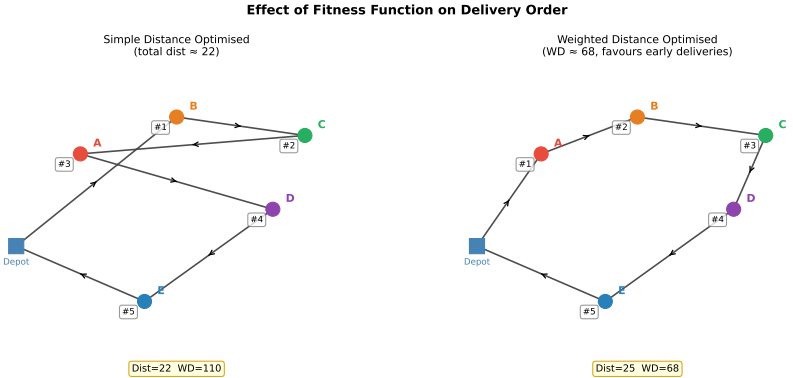


Figure: How the fitness function changes delivery order. *Left:* optimising simple total distance may leave nearby customers until last. *Right:* optimising weighted distance forces the algorithm to serve nearby customers first, reducing accumulated waiting.

10.2 Fitness: Choosing the Right Objective Function XII

Key Insight: Fitness Shapes Behaviour

The choice of fitness function is not a technical detail — it **determines what behaviour the algorithm is rewarded for**. Using total distance alone would produce routes that minimise mileage but may keep customers waiting for hours. Weighted distance aligns the algorithm's objective with the actual goal: serve customers as promptly as possible.

10.3 Implementation Issues: Real-World Complications I

Translating the mathematical model into a working software application requires careful engineering decisions that go beyond the algorithm itself. This section describes the architecture of the demonstration application and the practical challenges encountered.

Three-Layer Architecture

The application is built in three clearly separated layers, following the software engineering principle of *separation of concerns*:

1. **Imported base classes** — reusable components from earlier chapters: population management, route data structures, geospatial utilities, and file I/O.
2. **Food-specific classes** — subclasses whose names begin with `Food`: `FoodPopulation`, `FoodRoute`, `FoodEvaluator`. These specialise the evaluation function and genetic operators for the food-delivery problem.
3. **Application layer** — a single class called `FoodFacade`, implemented using the *Facade* design pattern. It exposes a clean, simple API to the calling application, hiding all optimisation details.

10.3 Implementation Issues: Real-World Complications II

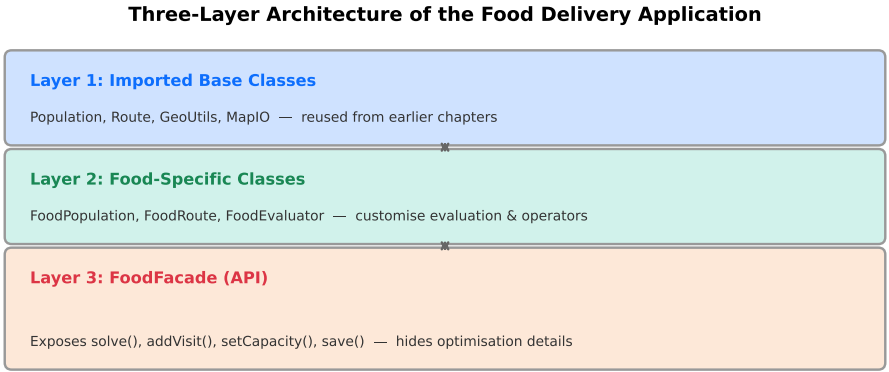
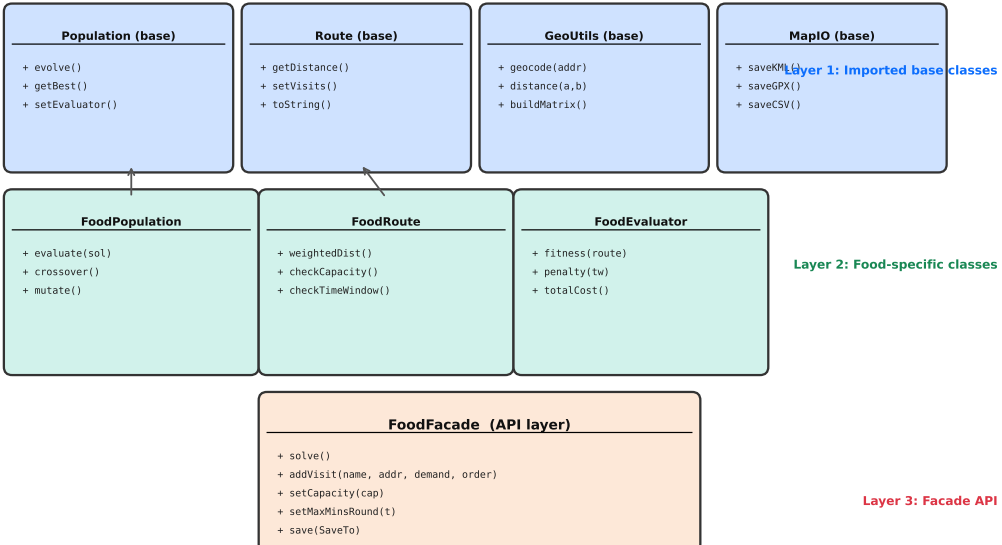


Figure: The three-layer architecture. Layer 1 is reused from earlier chapters; Layer 2 extends it for food delivery; Layer 3 (the Facade) is the only component visible to the calling application.

10.3 Implementation Issues: Real-World Complications III

Class Architecture: Food Routing Application (Three Layers)



Java: Main Application (Listing 10.1)

The following code (Listing 10.1 from the book) shows the complete main application. The `FoodFacade` class hides all optimisation complexity: the developer needs only four lines of logic to load, solve, and save a food-delivery problem.

Listing 1: Listing 10.1: The food application (Java).

```java
public class FoodDeliveryApp {
  public static void main(String[] args) {
    // Load a problem from a .csv file
    FoodFacade food = loadProblem(new File("./problem.csv"));

    // Run the evolutionary optimiser (all complexity is hidden)
    food.solve();

    // Save the solution in several output formats
    try {
      food.save(SaveTo.CONSOLE);  // print to screen
      food.save(SaveTo.KML);      // Google Earth
      food.save(SaveTo.GPX);      // GPS devices
      food.save(SaveTo.CSV);      // spreadsheet
    } catch (Exception e) {
      e.printStackTrace();
    }
  }
}
```

Java: Loading and Parsing the CSV (Listing 10.2)

The method `loadProblem()` reads the CSV file line by line and dispatches each line to `processLine()`, which inspects Column A and calls the appropriate `FoodFacade` setter. This design deliberately separates *data parsing* from *optimisation logic*: the `FoodFacade` class has no knowledge of the file format.

Listing 2: Listing 10.2: Problem loading and CSV parsing (Java).

```
private static FoodFacade loadProblem(File file) {
  // Create a new FoodFacade and populate it from the file
  try {
    FoodFacade food = new FoodFacade();
    Scanner myReader = new Scanner(file);
    while (myReader.hasNextLine()) {
      processLine(food, myReader.nextLine());
    }
    myReader.close();
    return food;
  } catch (FileNotFoundException e) {
    System.out.println("An error occurred.");
    e.printStackTrace();
    return null;
  }
}

private static void processLine(FoodFacade hf, String line) {
  // Process a single line of the .CSV file
  String[] data = line.split(",");
```

Python: Equivalent CSV Loader

Below is a Python equivalent of the Java CSV loader above, demonstrating how the same logic would be implemented in Python. This is useful for understanding the algorithm independent of the book's Java implementation.

Listing 3: Python equivalent of the food delivery CSV loader.

```
import csv

class FoodFacade:
    """Facade that loads, solves, and saves food delivery problems."""

    def __init__(self):
        self.reference = ""
        self.start_address = ""
        self.start_datetime = ""
        self.capacity = 30          # default: 30 bags
        self.mins_per_delivery = 5 # default: 5 minutes per stop
        self.max_mins_round = 500  # default: 500 minutes per round
        self.mode = "car"
        self.visits = []            # list of (name, address, demand, order)

    def add_visit(self, name, address, demand, order=""):
        self.visits.append({
            "name": name, "address": address,
            "demand": demand, "order": order
        })
```

Python: Weighted Distance Fitness Function

The weighted-distance fitness function is the core of the algorithm. This Python implementation computes $WD = \sum_i d_i \times wc_i$ given a list of pairwise distances and a route (list of customer indices).

Listing 4: Python: computing the weighted-distance fitness.

```python
import numpy as np

def weighted_distance(route: list, dist_matrix: np.ndarray) -> float:
    """
    Compute the weighted-distance fitness of a route.

    Parameters
    ----------
    route       : list of customer indices, e.g. [0, 2, 1, 4, 3]
                  (depot is represented by index -1 or implicitly at
                   the start and end)
    dist_matrix : 2-D numpy array where dist_matrix[i][j] is the
                  distance from location i to location j.
                  Index 0 is assumed to be the depot.

    Returns
    -------
    WD : float -- weighted distance (lower is better)
    """
    n_customers = len(route)
    full_route = [0] + route + [0]    # 0 = depot at start and end
    wd = 0.0
```

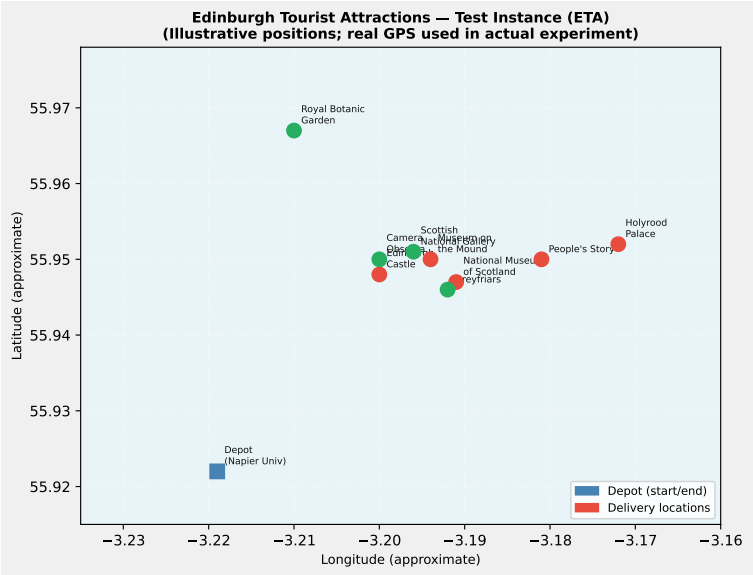
10.4 Results: Edinburgh Tourist Attractions Case Study I

To evaluate the algorithm, the authors created a benchmark problem instance based on real Edinburgh tourist attraction addresses. This is called the **Edinburgh Tourist Attractions (ETA) instance**.

Why a Public Test Instance?

The original problem instances used with real clients were commercially confidential and could not be published. The ETA instance uses publicly available addresses and provides a reproducible benchmark for comparing algorithm settings. The instances in the study had fewer than 100 deliveries each (in practice, fewer than 20 for many real clients).

10.4 Results: Edinburgh Tourist Attractions Case Study II



10.5 Conclusions I

This chapter has demonstrated a complete, deployable solution to the food-delivery variant of the Vehicle Routing Problem. Several important lessons emerge.

1. Package: Reuse the Algorithm as a Library

The evolutionary optimiser is packaged as a *JAR archive* (Java ARchive) — a compiled library that any Java application can import. Source code need not be exposed. This packaging approach is widely applicable: algorithm developers can ship their code as a library, and application developers integrate it without understanding the internals. Python equivalents include distributing code as a wheel (`.whl`) package on PyPI.

10.5 Conclusions II

2. Configure: Manage Execution Time

Waiting for an algorithm to finish is unacceptable in interactive applications. Providing configurable timeout values and stop-and-resume controls ensures that the user always receives an answer within a predictable time. The specific timeout values (and other parameters such as population size and mutation rate) are stored in a properties file, so a system administrator can tune them without modifying or recompiling the application.

3. Multiple Runs: Compensate for Randomness

Because evolutionary algorithms are stochastic (they use random numbers), a single run may find a solution that is better or worse than typical. Running the algorithm several times and presenting the best result across runs dramatically increases the probability of finding a high-quality solution. The multiple-run strategy is especially effective when combined with a fixed timeout: each run is allocated a fraction of the total budget.

10.5 Conclusions III

4. Fitness Function: Weighted Distance over Simple Distance

Using *weighted distance* as the fitness function rewards the algorithm for making early deliveries. Feedback from real users confirmed that this is indeed what they expected: customers who receive their delivery early in the route are more satisfied, even if the total route length is slightly longer.

10.5 Conclusions IV

Broader Takeaway: Algorithm + Engineering

A powerful algorithm alone is not sufficient for real-world impact. Successful deployment requires:

- A **well-designed fitness function** that captures true user preferences (not just a proxy like total distance).
- A **clean software architecture** (Facade pattern, layered design) that separates the optimiser from the application.
- **Practical termination strategies** that guarantee timely answers to users.
- A **configurable interface** so that domain experts (not just algorithm researchers) can tune and deploy the system.

Food Delivery as a Template

The architecture described in this chapter — base library $\rightarrow$ domain-specific subclasses $\rightarrow$ Facade API — is reused in the following two chapters for letter and parcel delivery. The same pattern can be applied to any combinatorial optimisation problem: vehicle routing, staff scheduling, facility location, etc.

Chapter 10 Summary

What We Covered

- Food delivery as a constrained VRP: capacity Q, max transit time, min delivery time
- Weighted distance $WD = \sum d_i \times wc_i$ as a user-aligned fitness function
- Three-layer software architecture with Facade pattern
- CSV input format and Java/Python parser
- Stop-and-Resume vs. Timeout termination
- ETA case study: effect of capacity and time constraints

Key Formulas

Simple distance:

$$F_{\text{dist}} = \sum_{i=1}^{n} d_i$$

Weighted distance:

$$WD = \sum_{i=1}^{n} d_i \times wc_i$$

where $wc_i =$ (customers left at leg i).

For 5 customers, route Dep→A→B→C→D→E→Dep: $F_{\text{dist}} = 25$, $WD = 86$.

References

- **Amos, D., Calinescu, R., et al. (2022).** *Nature Inspired Optimisation for Delivery Problems*. Springer, Chapter 10: Delivering Food, pp. 209–221.
- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).** *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. [Facade pattern, p. 185.]
- **Clarke, G., & Wright, J.W. (1964).** Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4), 568–581. [Foundational VRP paper.]
- **Toth, P., & Vigo, D. (Eds.) (2002).** *The Vehicle Routing Problem*. SIAM, Philadelphia.